CS 432 – Databases

Course Project / Assignment 4

Sharding of the Developed Application

Sujal Patel (22110261) Siddharth Doshi (22110250) Viraj Vekaria (22110287)
Nishit Mistry (22110172) Rutvi Shah (22110227)

Semester II (2025–2026)

GitHub Repository	https://github.com/SujalAVMP/SafeDocs
Video Link	https://youtu.be/R7N0pSnlGn8

1 Introduction

This assignment extends the SafeDocs application with horizontal partitioning through sharding. The existing application from Assignments 2 and 3 already provided a working Flask UI/API layer, RBAC, transactional document operations, and concurrency testing. The goal in Assignment 4 is to partition the data so that the application no longer treats all document records as if they belong to a single monolithic table.

Our implementation focuses on the **document workload** because that is the part of the application where the most natural single-record routing already exists. We sharded the `Document` data and the matching document-activity log data, then modified the application so that:

- single-document lookups and updates route directly to one shard,
- new document inserts are placed in exactly one shard,
- list and range-style document queries fan out across shards and merge their results, and
- dashboard / portfolio recent activity is reconstructed from sharded access-log tables.

The repository now supports two execution modes for the same routing logic:

- **local logical-table mode**: useful for development and reproducible testing on one MySQL instance
- **remote three-database mode**: uses the actual shard endpoints provided in the assignment handout

The final verification reported below was performed against the real shard host `10.0.116.184` on ports `3307`, `3308`, and `3309`.

2 Shard Key Selection and Justification

2.1 Chosen shard key

The shard key used in this implementation is `DocumentID` with a **hash-based** partitioning rule:

$$\texttt{shard_id} = \texttt{DocumentID} \bmod 3$$

2.2 Justification

High cardinality. `DocumentID` is a primary key and is unique for every document, so it naturally provides a large number of distinct values. This makes it suitable for hash-based distribution and avoids hot ranges that can appear with range sharding.

Stable. `DocumentID` is assigned once at insert time and never changes afterwards. This satisfies the stability requirement and prevents expensive reshuffling of rows between shards during normal updates.

Query-aligned. The existing application already performs many direct document operations using `DocumentID`, such as:

- `GET /api/documents/<id>`
- `PUT /api/documents/<id>`
- `DELETE /api/documents/<id>`
- document edit page routing using `/documents/edit/<id>`

For these operations, `DocumentID` enables exact routing to one shard without broadcasting.

2.3 Why not shard by MemberID or FolderID?

We considered member-centric sharding because portfolio queries and uploader counts are member-related. However, the application's most common write path is still direct document CRUD by document identifier. If we had sharded by `UploadedBy` or `FolderID`, then `/api/documents/<id>` would require either a directory lookup or a fan-out read. Using `DocumentID` keeps the most critical document CRUD path simple and deterministic.

2.4 Expected distribution and skew

For the migrated legacy dataset, the document distribution across three shards was:

Shard	Migrated documents	Rule
0	676	`DocumentID % 3 = 0`
1	676	`DocumentID % 3 = 1`
2	678	`DocumentID % 3 = 2`

This is acceptably balanced for a logical simulation. The slight difference of two records is expected and does not indicate skew.

3 Partitioning Strategy and Shard Layout

3.1 Chosen strategy

We implemented **hash-based sharding**. Hashing on the document primary key is simple, deterministic, and easy to compute in the Flask router. It also avoids the manual boundary management required by range-based partitioning.

3.2 Physical shard layout

The implementation uses the same logical partitioning rule in two physical layouts:

- **Local simulation mode**: shard tables named `shard_0_document`, `shard_1_document`, `shard_2_document`, and similarly for `AccessLog`

- **Remote deployment mode**: each physical shard database contains one `Document` table and one `AccessLog` table, and the application chooses the correct database connection using the same `DocumentID % 3` rule

  In addition, two helper tables were introduced:

- `DocumentIdSequence`: reserves globally unique document identifiers before insert routing
- `DocumentShardDirectory`: records the shard chosen for each document and whether it came from migration or a live routed insert

3.3 Why AccessLog was sharded too

Document activity is keyed by `DocumentID`, and the dashboard/portfolio views already depend on document activity. Keeping document rows and document-activity rows colocated on the same shard reduces cross-shard coupling for per-document behaviour and keeps activity routing consistent with document routing.

4 Data Migration

The legacy tables `Document` and `AccessLog` were not dropped. Instead, they were used as the source for one-time migration into the new shard tables. This choice preserved the original Assignment 1/2 dataset for validation.

The repository contains two migration entry points:

`Module_B/sql/sharding_setup.sql` (local table simulation)
`Module_B/remote_sharding_setup.py` (remote three-database deployment)

The core migration rule is the same in both cases:

```
target_shard = DocumentID % 3
```

Rows from `Document` and `AccessLog` are copied only to the shard selected by that hash rule. The same rule is also used later for all routed inserts, updates, deletes, and lookups.

4.1 Migration verification

We verified that no legacy rows were lost during migration:

Dataset	Legacy table count	Migrated rows found on shards
Document	2030	2030
AccessLog	15041	15041

This confirms that the migration preserved all existing document and document-activity rows without loss.

5 Query Routing in the Application

5.1 Implementation scope

The sharding router was added to the Flask application in:

`Module_B/app/app.py`

This assignment implementation shards the **document subsystem** of the application. Member management, authentication, folders, departments, roles, and security logs remain unsharded because they are global control-plane data in the current application design.

5.2 Single-key lookups

For lookups such as `GET /api/documents/<id>`, the shard is computed directly:

```
shard_id = DocumentID % 3
```

The router then queries only the correct shard backend: either one local shard table or one remote shard database.

5.3 Insert routing

For `POST /api/documents`, the application first reserves a new `DocumentID` using `DocumentIdSequence`, computes the shard, then inserts the new row only into the correct `Document` storage target for that shard.

The routed insert is also recorded in `DocumentShardDirectory` and the matching sharded access-log table.

5.4 Update and delete routing

Document updates and soft deletes use the same shard calculation on `DocumentID`. This preserves the existing Assignment 2/3 CRUD semantics while ensuring the document is modified in exactly one location.

5.5 Cross-shard list and range queries

List and range-style document queries use fan-out and merge:

- `GET /api/documents?title=...` queries all three document shards and merges the matches
- `GET /api/documents?id_start=... & id_end=...` queries all three shards, because a hash-based document-ID range can live on any shard
- the merged key-range result is ordered by `DocumentID`

5.6 Dashboard and portfolio activity

Recent activity is reconstructed using a `UNION ALL` query across the three sharded access-log tables. This preserves the user-facing behaviour of the dashboard and portfolio pages after sharding.

6 Verification Results

6.1 Document routing correctness

The routed API behaviour was verified using:

`Module_B/sharding_demo.py`

Representative checks that passed were:

- **Single-key lookup**: `/api/documents/1` returned the row from shard 1
- **Cross-shard range query**: `/api/documents?id_start=1&id_end=12` returned documents spanning shards 0, 1, and 2
- **Insert routing**: a newly created document was assigned a global ID, routed to one shard, and verified by direct SQL to exist on exactly one physical shard table

6.2 Remote shard verification

The same demo was also executed against the actual shard host supplied in the assignment handout using:

- host `10.0.116.184`
- ports `3307`, `3308`, and `3309`
- database/user `Group_Safe_Docs`

The remote shard state matched the expected hash partitioning exactly:

Metric	Shard 0	Shard 1	Shard 2
Document rows	676	676	678
AccessLog rows	5023	5077	4941
Wrong-shard documents	0	0	0
Wrong-shard access logs	0	0	0

We then ran the application in remote-shard mode and verified that:

- the migrated totals still matched the legacy dataset exactly (`2030` documents and `15041` logs),
- single-key lookups hit the correct remote shard,
- range queries still fan out and merge correctly,
- a live inserted document was placed on exactly one remote shard and then cleaned up after verification.

6.3 Routed insert integrity

Live post-migration inserts were also verified. Each newly created document:

- received a globally reserved `DocumentID`,
- was routed to one shard using the same hash rule,
- appeared in exactly one physical shard table, and
- was recorded in `DocumentShardDirectory`.

Because the demo can be executed multiple times, the exact total number of post-migration routed inserts may increase over time. The important invariant is not the absolute running total, but the fact that every new document lands on exactly one shard and can still be retrieved correctly by its document ID afterwards.

6.4 Regression check against Assignment 3

After sharding, the previous concurrency harness from Assignment 3 was executed again. The following still passed successfully:

- rollback integrity on a failed multi-step member-creation request
- concurrent delete race on the same document with exactly one successful delete and the rest failing cleanly
- mixed read-heavy API load with all requests completing successfully

This matters because it shows that the new shard router did not break the transactional behaviour and concurrency controls implemented earlier.

7 Sharding Approach and Isolation

7.1 Which approach was used in the final repository?

The final repository supports **both** approaches requested by the assignment:

- **logical shard tables** inside one MySQL database for local development and easy reproduction
- **multiple databases on different shard ports** for the actual deployment path from the shard-details handout

This makes the solution robust in two ways. First, it remains easy to run and inspect locally. Second, it can be pointed at the real three-shard deployment without changing the routing logic.

7.2 What isolation do we actually get?

In local-table mode, the shard tables are only logically isolated. In remote-database mode, the sharded `Document` and `AccessLog` data are physically separated across three database endpoints. However, this is still not a fully production-grade distributed system:

- all three remote shards are still part of one instructor-provided cluster environment,
- coordinator metadata such as `DocumentIdSequence` and `DocumentShardDirectory` remains centralized,
- there is no replication or automatic failover.

Therefore, the project now goes beyond a pure table-based simulation, but it still intentionally stops short of a replica-backed production distributed database.

8 Scalability and Trade-offs Analysis

8.1 Horizontal vs. vertical scaling

Vertical scaling would mean increasing the resources of one database server. Sharding instead partitions the data across multiple fragments so that growth can be handled by spreading records out. In the final remote mode, those fragments are already deployed as three separate database endpoints rather than just three tables.

8.2 Consistency

Single-document writes are straightforward because each document belongs to one shard. However, in remote mode, the application updates both the target shard and centralized coordinator metadata. There is no distributed two-phase commit across coordinator and shard connections, so a crash at the wrong moment could leave those two locations temporarily inconsistent until repaired.

8.3 Availability

Single-key document lookups depend on only one shard, which is a strength of sharding. However, list views, title filters, dashboard recent activity, and range queries currently fan out across all shards. If one shard becomes unavailable, those cross-shard queries would fail in the current implementation because the router does not yet provide degraded partial-result behaviour.

8.4 Partition tolerance

This implementation does not include distributed failure handling or replica failover. If a shard becomes unreachable, the current application surfaces that failure rather than continuing with stale or partial data. This is acceptable for the assignment because the trade-off analysis was requested as written analysis rather than additional failure-handling code.

8.5 Cost of the chosen shard key

The biggest trade-off of using `DocumentID` is that some useful queries do *not* align with the shard key. Examples include:

- exact title filters,
- dashboard-wide recent activity,
- portfolio activity by member,
- document-ID range queries under hash partitioning.

These queries require fan-out and merge. In other words, the chosen key optimizes direct CRUD routing better than it optimizes analytical or secondary-filter queries.

9 Observations and Limitations

- The sharded scope is intentionally limited to the document subsystem that the current Flask application actively uses.
- Related but currently unused tables such as `DocumentVersion`, `Permission`, `DocumentTag`, and `SharedLink` remain unsharded. Extending the shard design to those tables would require additional colocated routing rules.
- The shard directory is currently useful mainly for verification, insert bookkeeping, and future extensibility.
- The final code proves both a local logical simulation path and a real three-endpoint deployment path, but it does not attempt online rebalancing, replica management, or distributed commit recovery.

10 Conclusion

The Assignment 4 solution now satisfies both the conceptual and deployment aspects of sharding. We selected a stable, high-cardinality key, partitioned the existing document workload across three shards, migrated the legacy dataset without loss, and modified the application so that direct document operations route to one shard while range/list queries correctly fan out across all shards.

The repository can be demonstrated locally with logical shard tables and also run against the actual instructor-provided three-shard deployment. The design makes the trade-offs visible: single-document CRUD becomes simple and deterministic, but secondary filters and global activity views require cross-shard aggregation, and coordinator metadata remains a centralized dependency. These are realistic distributed-database trade-offs and align well with the educational goal of the assignment.